

CS543 Web Services — Assignment 3

Integrate an External SOAP Partner

Group ID: 02

Team Member	Roll Number
AMAN KUMAR	20251651017
SHRIKANTA NAYAK	20251651090
ADITYA VERMA	20251651009
PRATIK SINHA	20251651071

1. Context

CampusEats must charge students' cards through an external payment partner. For this design, the selected partner is **UniPay Bank** and the integrated operation is **charge**. CampusEats' own services remain REST-shaped, while SOAP is used only at the external enterprise edge because the partner controls the protocol and exposes a strong typed WSDL contract. A bank-style SOAP interface is appropriate because the tutorial identifies strong contracts, message-level security through WS-Security, and transaction/reliability requirements as reasons for using SOAP at this boundary.

Item	Decision
External partner	UniPay Bank Ltd.
CampusEats role	Consumer
Partner role	External payment provider
Operation	charge
Protocol	SOAP over HTTPS
Integration style	Synchronous request/response

2. WSDL Contract

The accompanying **partner.wsdl** is the contract that the CampusEats consumer binds to. It is a small, complete WSDL for the payment service. The abstract side describes what data and operation exist; the concrete side describes how the operation is transported and where the endpoint is located.

2.1 All Six WSDL Elements

WSDL element	CampusEats design	Purpose
types	cardToken, amount, currency; transactionId, status	XML Schema data definitions
message	chargeRequest, chargeResponse	Request/reply message definitions
portType	Payments / charge	Abstract operation interface
binding	PaymentsSOAP / SOAP over HTTP / document-literal	Concrete protocol and SOAPAction
service	PaymentsService	Named service grouping the endpoint
port	PaymentsSOAP	Binding + endpoint URL

2.2 WSDL-to-Call Mapping

The WSDL maps directly to the SOAP call: **types + message** determine the fields, **portType** defines the charge operation, **binding** specifies SOAP/HTTP and the SOAPAction, and **service/port** supplies the URL.

Contract item	Value
Operation	charge
Request fields	cardToken, amount, currency
Response fields	transactionId, status
SOAP style	document/literal

Contract item	Value
SOAPAction	urn:campuseats:payments/charge
Endpoint	https://api.unipay.example/pay

3. SOAP Message Design

The SOAP request is formed by hand to match the WSDL. Authentication is carried in the SOAP Header and the payment operation is carried in the SOAP Body. The accompanying files are **soap-request.xml**, **soap-response.xml**, and **soap-fault.xml**.

3.1 SOAP Anatomy

SOAP part	Content	Purpose
Envelope	SOAP envelope namespace	Outer message container
Header	UniPay authentication token	Carries partner credentials
Body	charge operation	Carries the business request
Fault	faultcode, faultstring, detail	Carries structured failure information

3.2 Request

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:uni="urn:campuseats:payments">
  <soap:Header>
    <uni:auth token="CE-MERCHANT-KEY"/>
  </soap:Header>
  <soap:Body>
    <uni:charge>
      <uni:cardToken>tok_9f2c</uni:cardToken>
      <uni:amount>300.00</uni:amount>
      <uni:currency>INR</uni:currency>
    </uni:charge>
  </soap:Body>
</soap:Envelope>
```

3.3 Success Response

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:uni="urn:campuseats:payments">
  <soap:Body>
    <uni:chargeResult>
      <uni:transactionId>tx_5521</uni:transactionId>
      <uni:status>captured</uni:status>
    </uni:chargeResult>
  </soap:Body>
</soap:Envelope>
```

3.4 SOAP Fault

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:uni="urn:campuseats:payments">
  <soap:Body>
    <soap:Fault>
      <faultcode>soap:Client</faultcode>
      <faultstring>Card declined</faultstring>
      <detail>
        <uni:error code="card_declined"/>
      </detail>
    </soap:Fault>
  </soap:Body>
</soap:Envelope>
```

The fault is structured rather than being treated as an ordinary broken response: the faultcode gives the category, faultstring gives the reason, and the detail contains the machine-readable partner code.

4. HTTP Binding

The SOAP envelope is the body of an HTTPS POST to the URL specified by the WSDL service/port. The SOAPAction value comes from the WSDL binding.

```
POST /pay HTTP/1.1
Host: api.unipay.example
Content-Type: text/xml; charset=utf-8
SOAPAction: "urn:campuseats:payments/charge"
```

```
<soap:Envelope> ... </soap:Envelope>
```

HTTP field	Value
Method	POST
Host	api.unipay.example
Content-Type	text/xml; charset=utf-8
SOAPAction	urn:campuseats:payments/charge
Endpoint	https://api.unipay.example/pay

5. End-to-End SOAP Flow

The interaction is synchronous. Orders sends the charge envelope to UniPay, UniPay verifies and processes the charge, and CampusEats waits for either a chargeResponse or a SOAP Fault before deciding whether the order can be confirmed.

```
CampusEats / Orders
|
| 1. POST charge SOAP envelope
v
UniPay Bank
|
| 2. Verify + charge
|
+-----> chargeResponse (success)
|
+-----> SOAP Fault (failure)
|
v
CampusEats / Orders
|
+-----> confirm order / return payment error
```

6. Discovery

CampusEats must obtain the UniPay WSDL before invoking the service. Two discovery approaches are possible: a direct WSDL URL supplied by the bank, or a catalogue/registry lookup. This submission uses the modern catalogue/registry form; no live UDDI server is required.

Registry field	Entry
Business	UniPay Bank Ltd.
Service	Payments
Endpoint	https://api.unipay.example/pay
Spec (tModel-style pointer)	urn:campuseats:payments — Payments WSDL

6.1 Discovery Lifecycle

```
1. UniPay publishes its WSDL
|
v
2. CampusEats finds the payment service in the catalogue/registry
|
v
3. CampusEats reads the WSDL and binds to the SOAP contract
|
v
4. CampusEats invokes the UniPay endpoint
|
v
5. Registry is no longer involved in the actual charge call
```

7. Fault Mapping into the CampusEats Contract

The partner's error vocabulary must not leak through the CampusEats service boundary. The Assignment 2 **placeOrder** contract already promised the caller a **payment declined** error. Therefore the integration adapter translates UniPay's **card_declined** fault into that CampusEats error.

External partner	CampusEats contract
card_declined	placeOrder → payment declined
Card declined message	Payment declined error returned to caller

This keeps UniPay-specific SOAP and fault vocabulary inside the integration boundary while callers continue to see the stable CampusEats contract.

8. Integration Boundary and Responsibility

The CampusEats Orders flow remains responsible for the business operation, while the UniPay integration adapter is responsible for translating between the internal CampusEats contract and the external SOAP contract. The adapter hides SOAP envelopes, authentication details, SOAPAction, endpoint details, and partner-specific faults from normal CampusEats callers.

Concern	Handled by
Order business decision	CampusEats Orders
Partner SOAP contract	UniPay integration adapter
Authentication header	UniPay integration adapter
SOAPAction / endpoint	UniPay integration adapter
Partner fault translation	UniPay integration adapter
Caller-visible error	CampusEats placeOrder contract

9. Design-Time Scope

This assignment is a design-time integration exercise. No running UniPay server or live payment gateway is required, and the work does not implement one of the CampusEats services. The WSDL, SOAP envelopes, HTTP binding, discovery record, and fault mapping together demonstrate how the external partner would be integrated at the contract level.